SYSTEM DESIGN · VOLUME I

Part II
Building Blocks of Scalable Systems

The pieces every scalable system is assembled from — clients and servers, load balancers, caches, gateways, and the two ways to grow.

CHAPTERS 6 – 10

The skeleton of a modern system, one building block at a time — from a single request’s journey to elastic, horizontally-scaled fleets.

CONTENTS

Part II at a glance

Five building blocks, each a chapter.

Part II assembles the blocks that scale *easily*. It ends by naming the block that does not — the data tier — which opens Part III.

CHAPTER 6

The Client–Server Model & the Request Lifecycle

Part II · Building Blocks — the skeleton almost every system is built on.

LEARNING OBJECTIVES

- Explain the client–server model and the roles each side plays.
- Describe the classic **3-tier architecture** (presentation, application, data) and why we separate tiers.
- Trace a request end-to-end through a modern multi-tier system, naming every hop.
- Explain the difference between **stateless** and **stateful** servers, and why stateless is the key to scaling.
- Identify which tier is easy to scale and which is the hard bottleneck — and why.

REAL-WORLD ANALOGY — A SERVICE DESK THAT GROWS INTO A LARGE OPERATION

Imagine a small shop with one service desk. You (the **client**) walk up and ask for something; a staff member (the **server**) fetches it from the records room (the **database**) and hands it back. As the shop gets popular, one desk isn't enough, so they add a **greeter** who directs each customer to whichever desk is free (a **load balancer**), and a **quick-answers board** for the most common questions so staff don't run to the records room every time (a **cache**). Nothing about the customer's request changed — but the operation behind the counter grew into tiers. That growth is exactly this chapter.

6.1 What Part II is about

Part I gave you the raw foundations: how one machine works, how machines talk, how the web works, and how to size a system with numbers. Part II now assembles those foundations into the **building blocks of scalable systems**. We start here with the most fundamental shape of all — clients and servers — and the path a request takes through the tiers. Every later block (load balancer, cache, gateway, database) plugs into this skeleton.

6.2 The client–server model

Almost every system you use follows one simple pattern: a **client** asks, a **server** answers. Your browser, phone app, or smart TV is the client; a machine in a data center is the server.

The client does…	The server does…
Shows the user interface; collects input; sends requests; renders responses.	Receives requests; runs the business logic; reads/writes data; sends responses.

Why is this split so universal? Because it gives **separation of concerns** and control: the server centralizes the logic and data, many clients share one authoritative system, and sensitive data stays behind the server where it can be secured — never handed to untrusted

clients. (Recall from Chapter 4 that HTTP requests are stateless — each one stands alone; that property will matter a lot in Section 6.6.)

THE ALTERNATIVE: PEER-TO-PEER

Not everything is client–server. In **peer-to-peer** (P2P) systems like BitTorrent, there is no central server — every participant is both client and server, talking directly to others. P2P shines for sharing load across users, but most business systems are client–server because they need a central, trusted authority over data. We'll focus on client–server for the rest of the book.

6.3 From one server to tiers

In Chapter 1 we saw the naive start: one machine running the app *and* the database together. It's simple but fails under growth — one point of failure, one hard limit, everything coupled. The fix is to split the system into **tiers**, each with one job, each able to scale and be secured on its own. The classic form is the **3-tier architecture**:

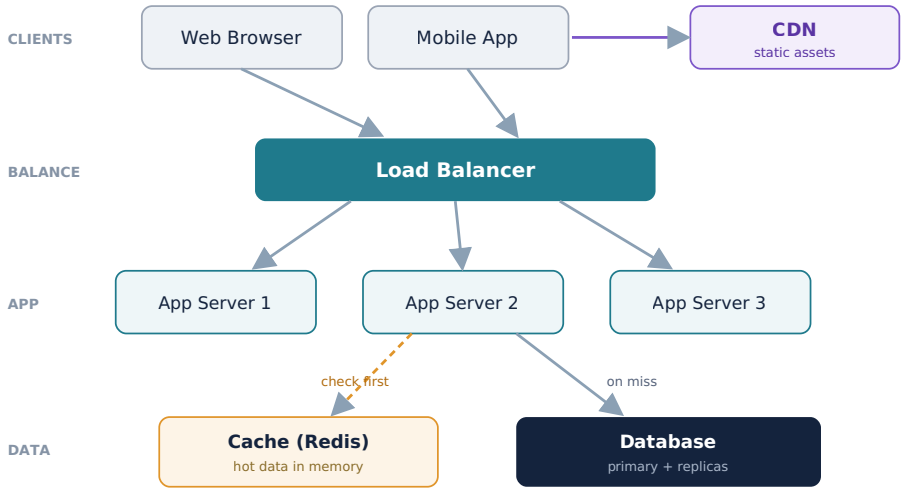
Tier	Job	Example
1. Presentation	What the user sees and interacts with.	Web/mobile frontend
2. Application (logic)	The business rules — the actual work.	Backend app servers
3. Data	Storing and retrieving data durably.	Database, object storage

KEY INSIGHT — TIERS LET EACH PART SCALE ON ITS OWN

Separating tiers means you can add app servers without touching the database, upgrade the database without redeploying the frontend, and put the data tier on a private network the internet can never reach directly. Each tier scales, deploys, and is secured independently. This separation is the foundation of every scalable design in this book.

6.4 A modern multi-tier request lifecycle

Here is what a real, scaled system looks like — the skeleton the next several chapters flesh out. Follow the arrows: this single figure contains the load balancer (Chapter 7), the cache (Chapter 8), and the CDN, all sitting on the client–server bones.



The request lifecycle, step by step

1. **Client sends a request** over HTTPS (DNS already resolved, Chapter 4). Static assets — images, CSS, JavaScript — are fetched from the **CDN** near the user, never from the app servers.
2. **The load balancer** is the public entry point. It picks a healthy app server to handle the request (Chapter 7).
3. **An app server runs the business logic.** Because app servers are *stateless* (Section 6.6), **any** of them can handle **any** request — which is exactly why we can have three, or three hundred.
4. **The app checks the cache first** (Chapter 8). A cache hit returns hot data from memory in well under a millisecond.
5. **On a cache miss, it queries the database** (the data tier), then usually writes the result back into the cache so the next request is fast.
6. **The response travels back** up through the load balancer to the client, which renders it.

This layered flow is the backbone of modern systems. The rest of Part II and Part III simply zoom into each box: the load balancer, the cache, the API gateway, horizontal scaling, and then the hard problem — scaling the data tier.

6.5 Stateless vs stateful servers

This is the most important idea in the chapter for scaling. It concerns whether a server **remembers** anything about a client between requests.

Stateless server	Stateful server
Keeps no client data between requests. Each request carries everything needed.	Remembers client data in its own memory (e.g. a login session stored locally).
Any server can handle **any** request → easy to add servers, easy failover.	A client's requests must return to the **same** server ("sticky sessions") → hard to scale, bad failover.
If one dies, another takes over instantly.	If it dies, the state it held is **lost**.

KEY INSIGHT — KEEP THE APP TIER STATELESS, PUSH STATE TO A SHARED STORE

The golden rule is to make app servers **stateless** and store any per-user state (like sessions) in a **shared** place all servers can reach — a database or a Redis session store. Then adding servers is trivial and a dying server loses nothing. This is the practical payoff of HTTP's statelessness from Chapter 4, and it's why real systems externalize sessions (Chapter 41) instead of keeping them in app-server memory.

PITFALL — STICKY SESSIONS

If you store a user's session in one server's memory, the load balancer must send that user back to that exact server every time ("session affinity" / sticky sessions). This undermines load balancing (traffic can't spread freely), breaks when that server dies, and blocks smooth scaling. Sometimes it's a pragmatic shortcut, but the durable answer is a stateless app tier with externalized state.

6.6 Where the tiers scale — and where it hurts

Tier	How hard to scale
Presentation (CDN + client)	Easy — the CDN serves static content from many edge locations.
Application (stateless app servers)	Easy — just add identical servers behind the load balancer.
Data (database)	**Hard** — you can't freely clone a database, because all copies must agree on the data.

KEY INSIGHT — WE MAKE THE APP TIER STATELESS TO ISOLATE THE HARD PROBLEM

Notice the pattern: presentation and application tiers scale almost for free, but the **data tier is the real bottleneck**, because data must stay *consistent* across copies. Keeping the app tier stateless is a deliberate move — it pushes all the hard scaling work down into one place, the data tier, which is exactly what Part III (replication, partitioning, sharding, consistency) is devoted to solving.

Common mistakes

WATCH OUT

- Storing sessions or other state in app-server memory — breaking horizontal scaling and failover.
- Exposing the database directly to clients or the public internet instead of hiding it behind the app tier.
- Coupling tiers so tightly they can't be scaled, deployed, or secured independently.
- Assuming the data tier scales like the app tier — ignoring that database copies must stay consistent.
- Routing static assets (images, CSS, JS) through app servers instead of a CDN.

- Sending traffic to dead servers because the load balancer has no health checks (Chapter 7).

Interview questions

1. Explain the client–server model and the 3-tier architecture. Why separate the tiers?
2. Trace a request from a browser all the way to the database and back in a modern architecture.
3. Why do we keep the application tier stateless? What specifically breaks if it’s stateful?
4. What are sticky sessions, and what problems do they cause?
5. Which tier is hardest to scale, and why?
6. Why shouldn’t clients talk to the database directly?
7. What is the difference between client–server and peer-to-peer architectures?

Hands-on exercises

1. Draw the 3-tier architecture for a simple blog app and label each tier’s responsibility.
2. For a “view profile” request, list every hop from client to database and back.
3. Take a system you know and classify its components as stateless or stateful.
4. Decide where session data should live so the app tier stays stateless, and justify it.
5. Sketch where a CDN fits in the lifecycle and which specific requests it should serve.

Mini project

TRACE AND TIER AN APP

Pick a simple app (say a to-do list). Draw the full modern request lifecycle: clients, CDN, load balancer, a stateless app tier, a cache, and the database. Then write out the exact numbered path taken by two requests — a `POST /todos` (create) and a `GET /todos` (list) — naming every hop and where the cache is checked. Mark which components are stateless and where any user state is stored. This cements the skeleton you’ll reuse for every design in the book.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE REFERENCE ARCHITECTURE

Using your Chapter 5 estimates, draw the **multi-tier reference architecture** for your platform: a load balancer, a stateless app tier, a cache, the database, and a CDN for media. Decide where user **session state** lives so the app tier stays stateless. Then annotate, based on your numbers, **which tier will need the most scaling** (almost certainly the data tier and the media/CDN path). Keep this as your base diagram — you’ll extend it with a real load balancer in Chapter 7, a caching strategy in Chapter 8, and database scaling in Part III.

Chapter summary

- The **client–server model** — clients ask, servers answer — is the universal shape of systems, chosen for separation of concerns, centralized data, and security.
- The **3-tier architecture** (presentation, application, data) splits a system so each tier can scale, deploy, and be secured independently.
- A modern request flows: **client → CDN (static) / load balancer → stateless app server → cache → database**, and back — the backbone the next chapters flesh out.
- **Stateless** app servers let any server handle any request, enabling easy scaling and failover; **stateful** servers force sticky sessions and lose state on failure. Externalize state to a shared store.
- Presentation and application tiers scale easily; the **data tier is the hard bottleneck** because copies must stay consistent — the subject of Part III.

COMING NEXT

Chapter 7 — Load Balancing & Reverse / Forward Proxies. We zoom into the box that made the app tier scalable. How does the load balancer choose a server? How does it know which servers are alive? What’s the difference between a reverse proxy and a forward proxy, and between Layer 4 and Layer 7 balancing? That’s next.

CHAPTER 7

Load Balancing & Reverse / Forward Proxies

Part II · Building Blocks — the traffic director that makes “add more servers” work.

LEARNING OBJECTIVES

- Explain what a load balancer is and the problems it solves.
- Compare the common load-balancing algorithms and know when to use each.
- Explain how **health checks** keep traffic away from dead servers and provide high availability.
- Distinguish **Layer 4** from **Layer 7** load balancing and their trade-offs.
- Distinguish a **reverse proxy** from a **forward proxy**, with real examples.
- Explain how to make the load balancer itself highly available, including across regions.

REAL-WORLD ANALOGY — THE RECEPTIONIST AT A LARGE OFFICE

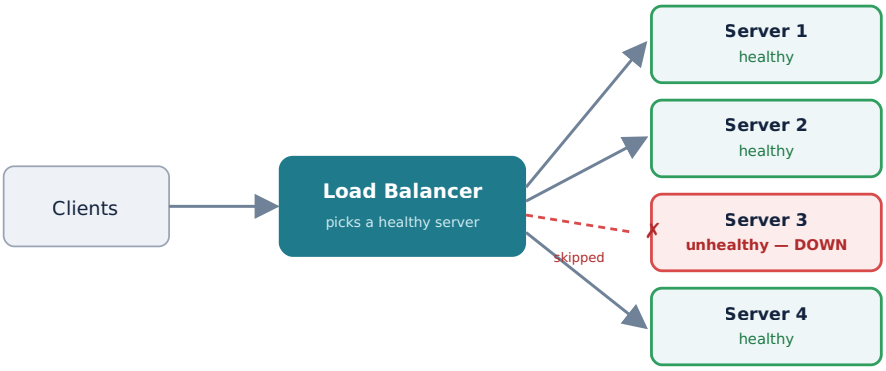
Picture a busy office with many staff members who can each help a visitor. A **receptionist** greets everyone at the door and sends each visitor to a staff member who is *free and present* — never to someone who called in sick that day (a **health check**). Visitors don’t need to know the office layout or who’s available; they just talk to the receptionist, who hides all of that and represents the whole office (a **reverse proxy**). A load balancer is exactly this receptionist for your servers.

7.1 The problem load balancing solves

In Chapter 6 we made the app tier **stateless** so we could run many identical app servers. But that raises an immediate question: when a request arrives, **which** server handles it? Without an answer, three things go wrong: clients would have to know every server’s address, one server could be swamped while others sit idle, and a crashed server would keep receiving requests that fail. A **load balancer** solves all three.

7.2 What a load balancer is

A load balancer sits between clients and your pool of servers. Clients see just one address (a virtual IP or DNS name); the load balancer spreads incoming requests across the healthy servers behind it and quietly removes any that fail. It is the component that turns “add more servers” from a wish into a working strategy — delivering both **scalability** (share the load) and **high availability** (route around failures).



Health checks let the load balancer detect Server 3 is down and route around it — the app tier keeps working despite the failure.

7.3 How it chooses a server: the algorithms

Algorithm	How it works	Best when
Round robin	Rotate through servers in order: 1, 2, 3, 1, 2, 3…	Servers are equal and requests are similar in cost.
Weighted round robin	Round robin, but bigger servers get a larger share.	Servers have different capacities.
Least connections	Send to the server with the fewest active connections.	Request durations vary a lot (some slow, some fast).
Least response time	Favor the server currently responding fastest.	You want to chase real-time performance.
IP hash	Hash the client’s IP to always pick the same server.	You need a client to stick to one server (affinity) without stored state.
Power of two choices	Pick two servers at random, send to the less loaded one.	You want near-optimal balancing with almost no coordination.

KEY INSIGHT — MATCH THE ALGORITHM TO THE WORKLOAD

Round robin is the sensible default, but it assumes every request costs the same. When some requests are far heavier than others, round robin can pile slow requests onto one unlucky server while others idle. **Least connections** adapts to that automatically. The algorithm is a small choice with a big effect on tail latency (Chapter 5) — pick it based on how uniform your requests really are.

7.4 Health checks: knowing who’s alive

A load balancer is only as good as its knowledge of which servers are healthy. Two mechanisms:

- **Active health checks:** the load balancer periodically probes each server (e.g. an HTTP `GET /health` every few seconds). If a server fails to respond correctly, it’s pulled from rotation until it recovers.

- **Passive health checks:** the load balancer watches real traffic; if a server starts erroring or timing out, it's marked unhealthy without a separate probe.

> **KEY INSIGHT — HEALTH CHECKS ARE WHAT DELIVER HIGH AVAILABILITY**
>
> Multiple servers alone don't give you high availability — you also need something to *notice* failures and route around them. Health checks are that something. Combined with a pool of stateless servers, they mean a single crashed app server is invisible to users. (The catch: the load balancer itself is now a critical component — Section 7.7.)

7.5 Layer 4 vs Layer 7 load balancing

Recall the network layers from Chapter 3. A load balancer can operate at two of them:

	Layer 4 (Transport)	Layer 7 (Application)
Works on	TCP/UDP — IP addresses and ports.	HTTP — URLs, headers, cookies.
Can it read the request?	No — it just forwards packets.	Yes — it understands the content.
Abilities	Fast, simple, protocol-agnostic passthrough.	Route by URL path or header, terminate TLS, sticky sessions by cookie, content-based routing.
Cost	Very low overhead.	Heavier — must parse (and often decrypt) each request.

So **L4 gives speed and simplicity**; **L7 gives intelligence**. For example, an L7 balancer can send `/api/*` to your API servers and `/images/*` to image servers, or route beta users by a header — things L4 simply cannot see. Most modern edges use L7 (which overlaps with the API gateway of Chapter 9); L4 is chosen when raw throughput or non-HTTP traffic (like a database connection) is the priority.

7.6 Reverse proxy vs forward proxy

A **proxy** is any intermediary that forwards traffic on someone's behalf. The chapter's two kinds differ in *whose* behalf:

```
FORWARD PROXY  (represents the CLIENT; sits on the client side)

  [ client ] \
  [ client ]  --> ( FORWARD PROXY ) --> internet --> server
  [ client ] /      hides & controls the clients
  uses: company web filter, monitoring, caching, anonymity

REVERSE PROXY  (represents the SERVER; sits on the server side)

                                    --> [ server ]
  client --> ( REVERSE PROXY ) -->  --> [ server ]
             fronts & hides servers --> [ server ]
  does: TLS termination, caching, compression, LOAD BALANCING
  examples: Nginx, HAProxy, Envoy  (a load balancer IS a reverse proxy)
```

KEY INSIGHT — THE MNEMONIC

A **forward** proxy stands in front of and represents the **client**; a **reverse** proxy stands in front of and represents the **server**. Your load balancer and API gateway are reverse proxies — they hide your servers, terminate TLS, and centralize cross-cutting concerns. A corporate internet filter is a forward proxy — it hides and controls the clients.

7.7 Making the load balancer itself reliable

We removed the single point of failure at the app tier — but now **all traffic flows through the load balancer**, so if it dies, everything is unreachable. You must make the balancer itself highly available:

- **Run more than one** load balancer — a redundant pair in active–passive (one on standby) or active–active (both serving).
- Use a **floating / virtual IP** or DNS failover so traffic shifts to the healthy balancer automatically if one fails.
- **Managed cloud load balancers** (AWS ELB/ALB/NLB, GCP, Azure) are redundant and self-healing by default — usually the simplest correct choice.

KEY INSIGHT — BALANCING HAPPENS AT MANY LEVELS

Load balancing isn't one box; it's a pattern that repeats. **Global** load balancing uses **GeoDNS** (Chapter 4) to send each user to their nearest healthy *region*; a **regional** load balancer then spreads traffic across app servers there; and you'll balance again across **database read replicas** (Part III) and between microservices (a service mesh, Chapter 30). Requests are balanced repeatedly on their way in.

Common mistakes

WATCH OUT

- Running with **no health checks** — happily sending traffic to dead servers.
- Deploying a **single** load balancer — recreating the single point of failure you tried to remove.
- Using **sticky sessions** unnecessarily (Chapter 6), which prevents even distribution.
- Choosing **round robin** when request costs vary wildly — use least connections instead.
- Terminating TLS at the balancer but leaving the balancer→backend hop unsecured in a zero-trust network.
- Confusing reverse and forward proxies, or assuming L7 features are free — they cost CPU (parsing and decryption).

Interview questions

1. What is a load balancer, and what problems does it solve?
2. Compare round robin, least connections, and IP hash. When would you choose each?
3. What is the difference between active and passive health checks?

4. Compare Layer 4 and Layer 7 load balancing. What can L7 do that L4 cannot, and at what cost?
5. Define reverse proxy and forward proxy, and give an example of each.
6. The load balancer is a single point of failure. How do you make it highly available?
7. How does global, multi-region load balancing work?
8. Why do sticky sessions complicate load balancing, and how do you avoid needing them?

Hands-on exercises

1. With 3 servers and 6 sequential requests, write out which server round robin selects each time.
2. Servers have active connection counts [5, 2, 8]. Which does “least connections” pick?
3. Decide L4 or L7 for each: routing `/api` vs `/static`; plain TCP passthrough to a database; routing by a request header.
4. Classify as reverse or forward proxy: Nginx in front of app servers; a corporate web filter; a CDN edge; a company’s outbound internet proxy.
5. Sketch a highly-available load-balancer setup using two nodes and a floating IP.

Mini project

PUT A LOAD BALANCER IN FRONT OF REAL SERVERS

Run two or three tiny web servers on different ports, each returning which port it is. Put **Nginx** (or HAProxy) in front as a reverse proxy load-balancing across them with round robin and a health check. Send repeated requests and watch the responses rotate between backends. Then **kill one backend** and confirm the load balancer stops sending it traffic while the others keep serving. You’ll have built, with your own hands, the exact behavior in this chapter’s hero diagram.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE BALANCING LAYER

Extend your Chapter 6 reference diagram with a real load-balancing layer. Choose an **algorithm** and justify it from your workload. Add **health checks**. Decide **L4 vs L7** (likely L7 at the edge, for path-based routing and TLS termination). Plan **high availability** for the balancer itself (redundant nodes or a managed cloud LB) and **global routing** (GeoDNS → regional load balancers) to match the multi-region ambitions in your estimates. Note where you’ll *also* balance across database read replicas in Part III.

Chapter summary

- A **load balancer** distributes requests across a pool of servers, giving both scalability and high availability, and presenting clients a single address.
- It chooses a server via an **algorithm** (round robin, least connections, IP hash…) — match it to how uniform your requests are.

- **Health checks** (active or passive) detect failed servers and route around them — the mechanism that actually delivers high availability.
- **Layer 4** balancing is fast, simple packet forwarding; **Layer 7** understands HTTP and can route by content and terminate TLS, at higher cost.
- A **reverse proxy** fronts servers (load balancers, Nginx, Envoy); a **forward proxy** fronts clients (corporate filters). Remember: reverse = server side, forward = client side.
- The balancer itself must be made **highly available** (redundant nodes, floating IP, or managed cloud LB), and balancing repeats at global, regional, and data-tier levels.

COMING NEXT

Chapter 8 — Caching: CDN, Redis, Memcached. We zoom into the “check cache first” box from Chapter 6 — the single highest-impact way to make a system fast. Where to cache, what to cache, how to keep caches fresh (invalidation), and the famous problems of stale data, thundering herds, and cache stampedes.

CHAPTER 8

Caching: CDN, Redis, Memcached

Part II · Building Blocks — the single highest-impact way to make a system fast.

LEARNING OBJECTIVES

- Explain what caching is, why it works, and what the cache hit ratio measures.
- Name the layers where caches live, from the browser to the database.
- Explain what a CDN does and why it slashes latency, origin load, and bandwidth cost.
- Compare Redis and Memcached and choose between them.
- Apply the main caching strategies (cache-aside, write-through, write-back, write-around).
- Handle the hard parts: invalidation, eviction, and failure modes like the thundering herd.

REAL-WORLD ANALOGY — THE LIBRARIAN'S FRONT CART

A librarian notices that a handful of books get requested constantly. Instead of walking deep into the stacks every time, she keeps those few on a cart right at the front desk. Most requests are now answered in seconds; only the rare, unusual request means a trip to the back. A **cache** is that front cart: a small, fast, close-by copy of the data people ask for most.

8.1 The problem caching solves

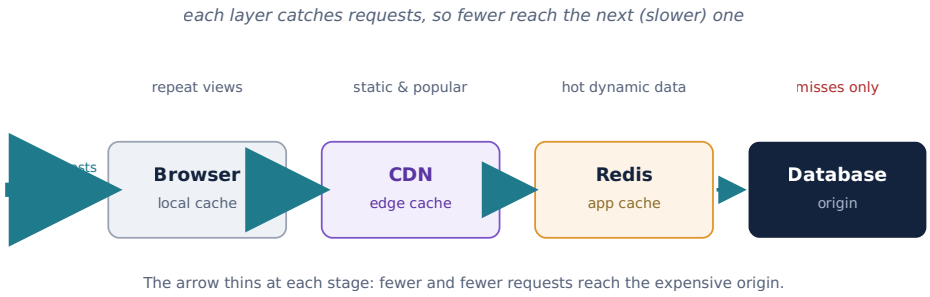
Chapter 2 taught that fetching data down the memory hierarchy is 100–1,000,000× slower, and Chapter 5 showed most real workloads are heavily **read-dominated** — the same data requested over and over. Recomputing a result or re-fetching a row from disk (or across the world) for every identical request is enormous waste. **Caching** stores a copy of frequently-used data somewhere fast and close, so most requests never pay the full cost.

8.2 Why caching works

Caching relies on **locality**: in almost every system, a small fraction of the data accounts for a large fraction of requests (the 80/20 rule; more formally a Zipf distribution). Cache that hot fraction and you serve most traffic from memory. The key metric is the **cache hit ratio** — the percentage of requests answered by the cache. A **hit** is fast; a **miss** falls through to the slower source. Even a 90% hit ratio means the slow path runs one-tenth as often.

8.3 Where caches live — the layers

Caching isn't one thing in one place; it's a series of layers along the request path. Each layer catches some requests so fewer reach the next, slower layer.



The arrow thins at each stage: fewer and fewer requests reach the expensive origin.

8.4 The CDN — caching at the edge

A **Content Delivery Network** caches content on servers spread around the world, so users are served from a nearby **edge** location instead of your distant origin. Recall from Chapters 4–5: a cross-world round trip costs ~150 ms, and image/video egress can be gigabytes per second. A CDN attacks all of that at once — cutting **latency** (content is physically closer), **origin load** (the origin barely gets touched), and **bandwidth cost**. CDNs use **pull** (fetch from origin on first request, then cache) or **push** (you upload content to the CDN ahead of time), and honor `Cache-Control` / TTL headers to decide how long to keep each item.

8.5 Redis vs Memcached

For caching your own dynamic data (not static files), the two classic in-memory stores are **Memcached** and **Redis**. Both keep data in RAM for microsecond access.

	Memcached	Redis
Data types	Simple key–value (strings)	Rich: strings, lists, sets, sorted sets, hashes, streams
Threading	Multi-threaded — simple and fast for pure caching	Mainly single-threaded command execution (very fast per core)
Persistence	None (pure cache)	Optional (snapshotting / append-only log)
Extras	Just caching	Pub/sub, atomic ops, TTLs, Lua scripting, can act as more than a cache
Choose when	You want the simplest possible cache for small values	You need data structures, atomic operations, or richer features

NOTE

Redis is the more common default today because its data structures and atomic operations are broadly useful (recall the atomic rate-limiter counter idea from the resilience patterns). Its licensing changed in 2024, prompting an open-source fork called **Valkey**; for design purposes they behave the same. Pick Memcached only when you truly need nothing beyond a plain key–value cache.

8.6 Caching strategies

“Use a cache” isn’t enough — you must decide *how* reads and writes interact with it.

Strategy	How it works	Trade-off
Cache-aside (lazy)	App checks cache; on miss, reads DB and populates cache. The default.	Simple; first request is a miss; risk of stale data until TTL.
Read-through	App asks the cache, which loads from the DB itself on a miss.	Cleaner app code; needs cache that supports it.
Write-through	Writes go to cache *and* DB together, synchronously.	Cache always fresh; writes are slower.
Write-back (write-behind)	Write to cache now, flush to DB later, in batches.	Very fast writes; risk of data loss if cache dies before flush.
Write-around	Writes go straight to DB, skipping the cache.	Avoids caching write-once data; reads of new data miss first.

Cache-aside is the most common: the application controls the cache explicitly, which is flexible and easy to reason about.

8.7 The hard part: invalidation and eviction

There is an old joke that the two hardest problems in computing are cache invalidation and naming things. Once you cache data, you must decide when the copy is **stale** and must be refreshed or removed.

- **TTL (time to live):** each entry expires after a set time. Simple and self-healing, but data can be stale for up to the TTL.
- **Explicit invalidation:** when the underlying data changes, delete or update the cached entry. Fresher, but you must remember to do it everywhere the data is written.

And when the cache fills up, an **eviction policy** decides what to drop: **LRU** (least recently used — the usual choice), **LFU** (least frequently used), or FIFO. The goal is to keep the hot data and shed the cold.

8.8 Cache failure modes to know

THUNDERING HERD / CACHE STAMPEDE

A very popular key expires, and suddenly *thousands* of requests miss at once and all slam the database together — which can crash it. Fixes: a **lock** so only one request recomputes while others wait; **request coalescing** (merge duplicate concurrent misses into one DB call); or **early/probabilistic expiration** so entries refresh before they all expire simultaneously.

CACHE PENETRATION & HOT KEYS

Penetration: requests for keys that don’t exist always miss and hammer the DB — fix by caching the “not found” result, or using a **Bloom filter** (Chapter 39) to reject unknown keys cheaply. **Hot key:** one key gets so much traffic it overloads a single cache node — fix by

replicating that key across nodes or caching it in the app itself. And beware the **cold start**: after a cache restart everything misses, so “warm” the cache with hot data before taking full traffic.

8.9 When NOT to cache

CACHING IS NOT FREE

A cache adds a moving part, memory cost, and the ever-present risk of serving stale data. Don’t cache data that changes on nearly every request (low reuse → low hit ratio), and be very careful caching data that must be **strongly consistent** and correct to the instant — like an account balance in a payment system. For such data, either don’t cache, or use write-through with tight invalidation. Caching trades a little consistency for a lot of speed; make that trade only where it’s safe.

Common mistakes

WATCH OUT

- Caching without an **invalidation plan**, then serving stale data forever.
- No protection against the **thundering herd** when a hot key expires.
- Not caching **negative results**, letting missing-key requests hammer the DB.
- Caching strongly-consistent data (balances, inventory counts) and showing wrong values.
- Ignoring the **hit ratio** — a cache with a low hit ratio adds cost for little gain.
- Forgetting **cold start** / warming after a cache restart or deploy.

Interview questions

1. What is caching and why does it work? What does the hit ratio measure?
2. Name the caching layers along a request’s path from browser to database.
3. Compare cache-aside and write-through. When would you use each?
4. Compare Redis and Memcached. When would you pick each?
5. What is a thundering herd / cache stampede, and how do you prevent it?
6. How do you handle cache invalidation? Compare TTL vs explicit invalidation.
7. When should you *not* cache something?

Hands-on exercises

1. For 100 requests with an 85% hit ratio, how many reach the database?
2. Decide a caching strategy for: a user’s profile (rarely changes), a live stock price (changes constantly), a shopping-cart total.
3. Design a cache key and TTL for “top 10 posts of the day.”
4. Sketch how you’d stop a stampede when the “home feed” cache entry expires.
5. List which items on a news homepage should come from a CDN vs an app cache vs the DB.

Mini project

ADD A CACHE AND MEASURE THE WIN

Take a small app with a slow endpoint (e.g. one that runs a heavy DB query). Add **Redis** using the **cache-aside** pattern: on each request, check Redis first; on a miss, run the query, store the result with a TTL, and return it. Measure the endpoint's latency and the **hit ratio** before and after warming up. Then delete the underlying data and observe the stale window until the TTL expires — invalidation made concrete.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE CACHING STRATEGY

Using your read-heavy Chapter 5 numbers, add caching to your reference architecture. Decide **what** to cache (user profiles, home feeds, post counts), the **strategy** (cache-aside for most), the **TTLs**, and the **invalidation** rules on writes. Put media behind a **CDN**. Identify your likely **hot keys** (a celebrity's profile) and your stampede risk (a viral post's feed entry), and note a mitigation for each. This caching layer is what will let your read QPS be served without melting the database in Part III.

Chapter summary

- **Caching** keeps a fast, close copy of frequently-used data; it works because of **locality** (a small hot fraction serves most requests). The **hit ratio** is the key metric.
- Caches form **layers**: browser → CDN → app/Redis → database; each catches requests so fewer reach the slow origin.
- A **CDN** caches at the edge, cutting latency, origin load, and bandwidth cost for static and popular content.
- **Redis** (rich data structures, optional persistence) vs **Memcached** (simple, pure cache) — pick by whether you need more than plain key–value.
- Choose a **strategy** (cache-aside is the default) and plan **invalidation** (TTL and/or explicit) and **eviction** (usually LRU).
- Guard against **thundering herds, penetration, hot keys, and cold starts**, and don't cache strongly-consistent, low-reuse data.

COMING NEXT

Chapter 9 — API Gateways & the Edge. We complete the "front door" of the system: a single smart entry point that routes, authenticates, rate-limits, and aggregates — building directly on the reverse proxy of Chapter 7.

CHAPTER 9

API Gateways & the Edge

Part II · Building Blocks — the single smart front door to your whole system.

LEARNING OBJECTIVES

- Explain what an API gateway is and the problems it solves.
- List the gateway's key jobs: routing, authentication, rate limiting, aggregation, TLS, and more.
- Explain request aggregation and the Backend-for-Frontend (BFF) pattern.
- Distinguish an API gateway from a load balancer and a service mesh.
- Know the gateway's trade-offs and when a separate gateway isn't worth it.

REAL-WORLD ANALOGY — THE BUILDING CONCIERGE

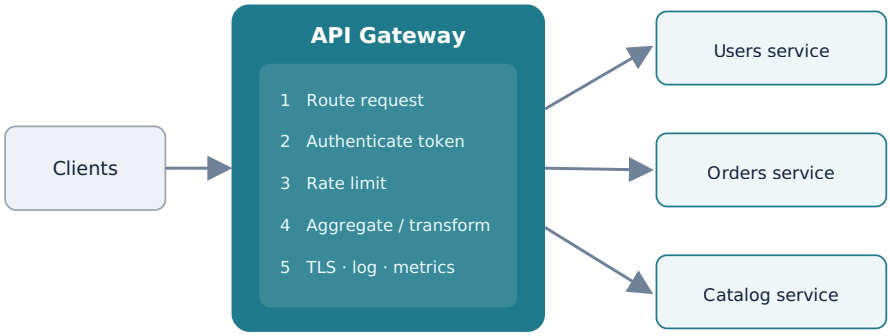
A large office building has one **concierge** desk that every visitor passes. The concierge checks who you are (authentication), stops anyone trying to flood the lobby (rate limiting), directs you to the right department (routing), and can even gather items from several offices so you don't walk the whole building yourself (aggregation). Visitors never wander the halls hunting for the right room. An **API gateway** is that concierge for all the traffic entering your system.

9.1 The problem the gateway solves

By now the system has grown: a load balancer, an app tier, caches — and soon many separate services (microservices, Part V). Without a single entry point, clients would have to know the address of every service, and each service would re-implement the same cross-cutting concerns: authentication, rate limiting, logging, TLS. That is duplicated, fragile, and couples clients to your internal structure. An **API gateway** is the single front door that centralizes all of this.

9.2 What an API gateway is

An API gateway is a server that sits at the **edge**, in front of all your backend services, and every client request goes through it. It is a specialized **reverse proxy** (Chapter 7) — but where a plain load balancer just distributes traffic, a gateway is **application-aware** and feature-rich, handling the cross-cutting concerns that every service would otherwise duplicate.



9.3 The key jobs of a gateway

Job	What it does
Routing	Sends each request to the correct backend service. Clients call one address; services can move or split without clients noticing.
Authentication	Validates the caller's token once, at the edge, rejecting bad requests before they reach any service (Chapters 5, 41).
Rate limiting	Caps how much any client can send, protecting backends from abuse and spikes (Chapter 35).
Aggregation	Turns one client call into several backend calls and merges the results — fewer round trips.
TLS termination	Handles HTTPS at the edge so internal services can work in plain HTTP on a trusted network.
Cross-cutting	Central logging, metrics, caching, request/response transformation, and protocol translation (e.g. external REST to internal gRPC).

KEY INSIGHT — DO CROSS-CUTTING WORK ONCE, AT THE EDGE

Auth, rate limiting, TLS, and logging are needed by *every* service. Without a gateway you copy that logic into all of them and maintain it many times over. The gateway's core value is centralizing these concerns in one place, so each service can focus purely on its business logic.

9.4 Aggregation and Backend-for-Frontend

Mobile networks make round trips expensive (Chapter 3). If a screen needs data from four services, making the phone call all four is slow. With **aggregation**, the client makes *one* call to the gateway, which fans out to the four services in parallel and returns a single merged response.

This is often specialized as a **Backend-for-Frontend (BFF)**: a separate gateway tailored to each client type — one for mobile, one for web, one for TV — each shaping and combining exactly what that interface needs, instead of forcing one generic API to serve all of them.

9.5 Gateway vs load balancer vs service mesh

These three are easy to confuse; they solve different parts of the traffic problem.

Component	Role
Load balancer	Distributes traffic across instances of a service (Chapter 7). Mostly “which server?”
API gateway	The smart **edge** front door for *incoming* client traffic (“north–south”): routing, auth, rate limiting, aggregation.
Service mesh	Manages traffic *between internal services* (“east–west”): mTLS, retries, tracing — usually via sidecars (Chapter 30).

A gateway often *contains* a load balancer (it must still pick a healthy instance of each backend). The mesh handles what happens once traffic is already inside.

9.6 Trade-offs and pitfalls

THE GATEWAY IS A CRITICAL COMPONENT

All traffic flows through it, so it’s a **single point of failure** and a potential **bottleneck** — always run several instances behind a load balancer, exactly as in Chapter 7. And resist stuffing business logic into it: a gateway that grows fat with domain rules becomes a **“distributed monolith”** front-end that every team must coordinate on, destroying the independence you built services to gain. Keep the gateway to cross-cutting concerns only.

9.7 When not to use a separate gateway

DON’T ADD IT BEFORE YOU NEED IT

A single, simple backend often doesn’t need a dedicated gateway — your load balancer and web framework may already handle TLS, routing, and auth adequately. The gateway earns its place when you have **many services** or **many client types**, or when cross-cutting concerns are genuinely being duplicated. Like every block in this book, add it when the numbers and the pain justify it, not by default.

Common mistakes

WATCH OUT

- Running a **single** gateway instance — recreating a single point of failure.
- Putting **business logic** in the gateway, turning it into a distributed monolith.
- Letting each service re-implement auth and rate limiting instead of centralizing at the edge.
- Confusing the gateway (edge, north–south) with the service mesh (internal, east–west).
- Adding a heavy gateway to a simple single-service app that doesn’t need one.
- Forgetting the gateway adds a network hop — keep its own processing lean.

Interview questions

1. What is an API gateway, and what problems does it solve?
2. List the key responsibilities of a gateway.
3. What is request aggregation, and what is the Backend-for-Frontend pattern?
4. How does a gateway differ from a load balancer and from a service mesh?
5. Why is the gateway a single point of failure, and how do you address that?
6. When would you *not* introduce a separate API gateway?
7. Why is doing authentication at the gateway better than in every service?

Hands-on exercises

1. List which cross-cutting concerns you'd move to a gateway for a 5-service system.
2. Design one aggregated endpoint that a mobile home screen could call instead of four separate calls.
3. For each, say gateway or service mesh: validating a user's login token; encrypting service-to-service calls; routing `/api/v2` to a new backend.
4. Sketch a highly-available gateway deployment.
5. Decide whether a simple single-backend blog needs a separate gateway, and justify it.

Mini project

STAND UP A GATEWAY

Using a gateway (or Nginx configured as one), put a single entry point in front of two small services — say a "users" service and a "posts" service. Configure **path-based routing** (`/users/*` and `/posts/*`), add a simple **rate limit**, and terminate **TLS** at the gateway. Then add one **aggregated** endpoint that calls both services and merges the result. You'll have built the concierge from this chapter's hero diagram.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE EDGE

Add an **API gateway** to your reference architecture as the single front door. Decide its responsibilities (routing to your services, token authentication, rate limiting, TLS termination), design one **aggregated** endpoint your mobile feed screen can call, and plan the gateway's own **high availability**. Note the boundary: the gateway handles incoming client traffic; internal service-to-service traffic will be handled by the mesh in Part V. Keep the gateway thin — cross-cutting concerns only.

Chapter summary

- An **API gateway** is the single, application-aware front door at the edge — a feature-rich reverse proxy that every client request passes through.

- Its key jobs are **routing, authentication, rate limiting, aggregation, TLS termination**, and other cross-cutting concerns — done **once** so services don’t each duplicate them.
- **Aggregation** and the **BFF** pattern cut client round trips by merging several backend calls into one.
- A gateway is the **edge** (north–south); a **load balancer** picks servers; a **service mesh** governs internal (east–west) traffic.
- Run **multiple** gateway instances (it’s a critical path) and keep it **thin** — no business logic — and only add it when you genuinely need it.

COMING NEXT — CLOSING PART II

Chapter 10 — Vertical vs Horizontal Scaling. We’ve added load balancers, caches, and a gateway — now we formalize the two ways to grow, revisit the single-machine ceiling, and set up the hardest scaling problem of all: the data tier, which opens Part III.

CHAPTER 10

Vertical vs Horizontal Scaling

Part II · Building Blocks — the two ways to grow, and the ceiling that forces the hard one.

LEARNING OBJECTIVES

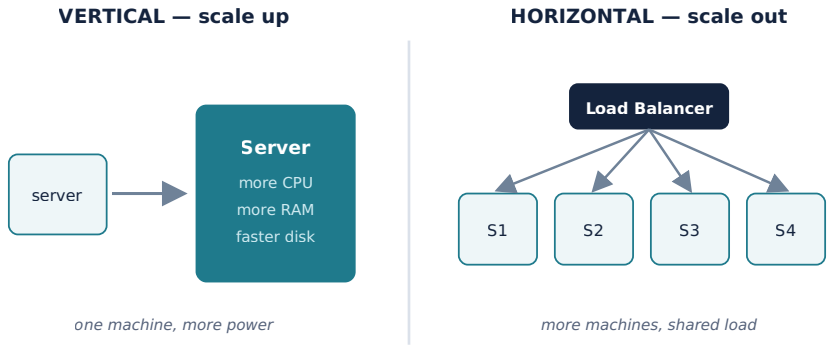
- Define vertical scaling (scale up) and horizontal scaling (scale out) and their trade-offs.
- Explain why every large system eventually scales out, and why statelessness is what makes it possible.
- Explain autoscaling and elasticity, and what they require.
- Identify why the data tier is the hard tier to scale — the bridge to Part III.
- Follow a pragmatic, step-by-step scaling path instead of over-building on day one.

REAL-WORLD ANALOGY — A BIGGER TRUCK VS MORE TRUCKS

A delivery company with too many parcels has two choices. It can buy one **bigger truck** that carries more (that's **vertical** scaling — a more powerful single machine). Or it can buy **more trucks** and split the deliveries among them (that's **horizontal** scaling — more machines sharing the load). The bigger truck is simple but there's a limit to how big a truck you can buy — and if it breaks down, everything stops. More trucks can grow almost without limit and survive one breaking down, but now you need a dispatcher to coordinate them. That dispatcher is your load balancer.

10.1 Two ways to grow

We've been building toward this since Chapter 1. When one machine can no longer keep up — too many requests, too much data — you scale. There are exactly two directions, and knowing when to use each is a core design skill.



10.2 Vertical scaling (scale up)

Give the single machine more power: more CPU cores, more RAM, a faster disk, more network bandwidth.

Advantages	Disadvantages

Simple — no code or architecture changes. No distributed-systems complexity. Strong consistency stays easy (one machine, one copy of data).	A hard **ceiling** — you can only buy so big. Cost grows **non-linearly** (top-end hardware is disproportionately expensive). Still a **single point of failure**. Often needs downtime to upgrade.

Vertical scaling is the right *first* move: it's cheap in engineering effort and buys real time. It's also often the pragmatic choice for databases, which are genuinely hard to distribute (Part III).

10.3 Horizontal scaling (scale out)

Add more machines and share the work across them, coordinated by a load balancer (Chapter 7).

Advantages	Disadvantages
Near-**unlimited** growth — just add machines. **No single point of failure** with several nodes. Uses cheap commodity hardware. Scales incrementally and **elastically** with demand.	Real **complexity**: needs load balancing, **stateless** servers (Chapter 6), and — the hard one — distributing data while keeping it consistent (Part III). Harder to operate and debug.

KEY INSIGHT — STATELESSNESS IS WHAT MAKES SCALE-OUT POSSIBLE

You can only add identical app servers freely because they're **stateless** (Chapter 6): any server can handle any request, so the load balancer can spread traffic and replace dead nodes at will. The moment a server holds per-user state in memory, scale-out breaks (sticky sessions, lost state). This is why "keep the app tier stateless, push state to a shared store" is the rule that unlocks horizontal scaling.

10.4 Autoscaling and elasticity

In the cloud, horizontal scaling becomes **elastic**: the system automatically adds instances when load rises and removes them when it falls. **Autoscaling** can be *reactive* (add servers when CPU or QPS crosses a threshold), *scheduled* (scale up before a known busy period), or *predictive* (forecast demand). This is how spiky traffic (Chapter 5) is served without paying for peak capacity around the clock.

WHAT AUTOSCALING REQUIRES

Elasticity only works if the pieces from earlier chapters are in place: **stateless** servers (so new ones can join instantly), **fast startup**, **health checks** and a **load balancer** (so new instances receive traffic and dead ones are dropped). Autoscaling is the payoff of doing the earlier building blocks correctly.

10.5 The hard tier: scaling data

KEY INSIGHT — APPS SCALE OUT EASILY; DATA DOES NOT

The stateless app tier scales out almost for free — add servers behind the load balancer. But you **cannot simply clone a database**, because every copy must agree on the data. Distributing data across machines while keeping it correct is *the* hard problem of large

systems, and it’s exactly what **Part III** is devoted to: **replication** (copies for reads and safety), **partitioning / sharding** (splitting data across machines for writes and size), and the consistency trade-offs (**CAP**) that come with them.

10.6 A pragmatic scaling path

Real systems don’t leap to a thousand shards on day one. They grow in steps, adding complexity only when the numbers demand it (Chapter 1’s warning against premature optimization):

```
1. One server                (simple; fine to start)
2. Scale UP  (bigger box)     -> cheap, buys time
3. Separate tiers            -> app / database / cache split (Ch 6)
4. Scale the app tier OUT    -> many app servers behind an LB (Ch 7)
                                + cache (Ch 8) + CDN
5. Scale reads               -> database read replicas (Part III)
6. Scale writes & data size  -> sharding / partitioning (Part III)

Move to each step only when the previous one is no longer enough.
```

THE JUDGMENT

Each step down this path adds capability *and* complexity. Jumping straight to step 6 for an app with a thousand users is over-engineering — pure cost, no benefit. The skill is reading your Chapter 5 numbers and moving exactly as far down the path as the load requires, and no further.

Common mistakes

WATCH OUT

- Reaching for horizontal scaling and sharding when a bigger machine would have done the job for years.
- Trying to scale out an app tier that still holds **state** in memory — it won’t work cleanly.
- Assuming the database scales like the app tier — ignoring that copies must stay consistent.
- Building autoscaling without stateless servers, health checks, or fast startup.
- Treating vertical scaling as “wrong” — it’s often the correct, cheapest first step.
- Forgetting that top-end vertical hardware costs rise non-linearly.

Interview questions

1. Define vertical and horizontal scaling. Give the main pros and cons of each.
2. Why do large systems eventually scale out rather than up?
3. Why is statelessness a prerequisite for horizontal scaling?
4. What is autoscaling, and what does it require to work?
5. Which tier is hardest to scale, and why?
6. Walk through a pragmatic scaling path from one server to a sharded system.

7. When is vertical scaling the *right* choice?

Hands-on exercises

1. For a service hitting its single server's CPU limit, list one vertical and one horizontal option and a trade-off of each.
2. Identify what must be true about your app servers before you can autoscale them.
3. Given traffic that is 10× higher for two hours each evening, describe an autoscaling policy.
4. Explain why you can add app servers freely but not database servers.
5. Place your capstone system on the six-step scaling path — which step does your Chapter 5 scale require?

Mini project

PROVE SCALE-OUT WITH A LOAD TEST

Run one instance of a small app behind a load balancer and use a load-testing tool to push requests until latency degrades (recall the utilization curve, Chapter 5). Record the throughput ceiling. Now add a second and third identical instance behind the same load balancer and repeat. Watch the throughput rise roughly with the number of instances — horizontal scaling, measured with your own hands. Note what would break if the app kept session state in memory.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE SCALING PLAN

Write your platform's **scaling plan** along the six-step path. Where do you scale up first? When do you scale the app tier out (and confirm it's stateless)? Define an **autoscaling** policy for your evening peak from the Chapter 5 estimates. Then mark the point where the plan hits the wall — the **data tier** — and note that replication and sharding (Part III) are the next tools you'll need. This plan is the natural handoff from Part II into Part III.

Chapter summary

- **Vertical scaling** (bigger machine) is simple and a good first step, but has a hard ceiling, rising costs, and remains a single point of failure.
- **Horizontal scaling** (more machines) grows almost without limit and removes single points of failure, at the cost of real distributed-systems complexity.
- Scale-out of the app tier works only because it's **stateless**; externalize state to a shared store.
- **Autoscaling** makes horizontal scaling elastic, but requires stateless servers, fast startup, health checks, and a load balancer.
- The app tier scales out easily; the **data tier is the hard problem** because copies must stay consistent — the subject of Part III.

- Follow a **pragmatic path**: up, then separate tiers, then app-tier out, then read replicas, then sharding — each step only when needed.

COMING NEXT — PART III BEGINS

Chapter 11 — Relational Databases & SQL. Part II gave you the building blocks that scale easily. Part III confronts the hard tier — data. We start at the foundation of storage: the relational model, SQL, and the guarantees that make databases trustworthy, before we learn to replicate and shard them.